Title: Student Course Registration & Fee Management System

SCENARIO (REALISTIC CONTEXT)

A training institute manages students enrolling in different courses.

Each student:

- registers for a course
- pays course fees
- can update details
- may cancel registration

The system must ensure:

- data consistency
- no duplicate registrations
- correct fee handling
- safe database operations (transactional integrity)

DATABASE DESIGN

You are given **two tables**:

```
student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    branch VARCHAR(50)
)
registration (
    reg_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT,
    course_name VARCHAR(50),
    fees_paid DOUBLE,
    FOREIGN KEY (student_id) REFERENCES student(id)
)
```

ARCHITECTURE REQUIREMENT

The application must follow **layered architecture**:

```
UI Layer (Menu / Main Class)
        ↓
Service Layer (Business Logic)
        ↓
DAO Layer (JDBC)
        ↓
Database
```

FUNCTIONAL REQUIREMENTS

1. Student Registration (Core Operation)

A student registers for a course.

Rules:

- Student must exist in `student` table
- Student cannot register for the same course twice
- Fee must be greater than 0

Critical Requirement:

- This must be handled using **transaction management**

First-Principles Thinking:

- What if insert fails?
- What if duplicate registration occurs?
- What should be atomic?

---

2. Add New Student

Add a new student to the system.

Constraints:

- ID must be unique
- Name should not be empty

- Age must be valid (>0)

---

3. View All Students with Registrations

Display:

```
Student Details + Course + Fees Paid
```

Requirement:

- Use JOIN query
- Handle students with no registrations

---

4. Search Student Registration by ID

Fetch:

- student details
- all registered courses

---

5. Update Student Details

Update:

- name
- branch

Constraint:

- Student must exist

---

6. Update Course Fee

Modify the `fees_paid` for a student's course.

Constraint:

- Fee must remain positive

---

7. Cancel Course Registration

Delete a student's registration for a specific course.

Rule:

- Only remove registration, NOT student

---

8. Delete Student (Critical Case)

Delete a student completely.

Constraint:

- First delete all registrations
- Then delete student

Critical Requirement:

- Must be done using **transaction**

---

9. Generate Report: High Paying Students

Display students who have paid more than a given fee.

---

10. Generate Report: Course-wise Count

Display:

```
Course Name → Number of Students
```

TRANSACTION REQUIREMENTS (MANDATORY)

You must implement transactions in:

Case 1:

Student registration

```
Insert into registration
```

If any step fails → rollback

Case 2:

Delete student

```
Delete from registration
Delete from student
```

If one fails → rollback

FIRST PRINCIPLE THINKING (MANDATORY)

For each operation, students must answer:

- What is the **input validation rule**?
- What is the **failure condition**?
- What should be **atomic**?
- What can cause **data inconsistency**?

NON-FUNCTIONAL REQUIREMENTS

- Use **PreparedStatement only**
- Use **try-with-resources**
- No SQL injection
- No direct DB calls in main class
- Proper exception handling
- Clear console output

EXPECTED CLASS STRUCTURE

```
com.project.app
 ├ model
 │  ├ Student.java
 │  └ Registration.java
 ├ dao
 │  ├ StudentDAO.java
 │  └ RegistrationDAO.java
 ├ service
 │  └ StudentService.java
 ├ util
 │  └ DBUtil.java
 └ app
    └ MainApp.java
```

MENU REQUIREMENT

```
1. Add Student
2. Register for Course
3. View All Students with Courses
4. Search Student by ID
5. Update Student
6. Update Course Fee
7. Cancel Registration
8. Delete Student
9. High Paying Students Report
10. Course-wise Student Count
11. Exit
```

EDGE CASES (MUST HANDLE)

- Duplicate student ID
- Duplicate course registration
- Negative fee
- Student not found
- Empty result sets
- DB failure during transaction